Design and Analysis of Algorithm

↳ Time complexity

↳ Space complexity

Time complexity → TC of an algorithm is the amount of computer time it needs to run to completion.

Space complexity → SC of an algorithm is the amount of m/s it needs to run to completion.

DESIGN METHODS:-

- → DAC (Divide & Conquer)
  - → Quick sort
  - → Merge sort
  - → Binary search
  - → Strassen's Matrix multiplication
- → Greedy Method
  - → Knap sack Problem
  - → Minimum spanning tree
    - ↓ Kruskal
    - ↓ Prims
  - → Single source shortest path
    - ↓ Dijkstra
    - ↓ Bellman ford
- → Dynamic programming
  - → All pair shortest path → Floyd Warshall
  - → 0/1 Knapsack problem
  - → Matrix chain multiplication
  - → Longest common subsequence
  - → Travelling sales person problem
- → Back-tracking
- → Branch & Bound
  - → 0-1 Knapsack problem
  - → Travelling salesperson problem
- → Hamiltonian cycle
- → Graph Coloring problem
- → N-Queen problem

Algorithm → It is a finite set of instructions that if followed accomplishes a particular task.

Characteristics of an algo:-

1) I/P → 0 or more qty. is externally supplied.

Scanned by CamScanner

2) O/P → Atleast 1 qty. is produced.

3) Effectiveness → Each instruction must be basic so that it can be carried out by a person using only pencil and paper.

4) Finiteness → If we trace out the instructions of an algo., then for all cases algo. should terminate after finite no. of steps.

5) definiteness → Each instruction should be clear & unambiguous.

Asymptotic Notations

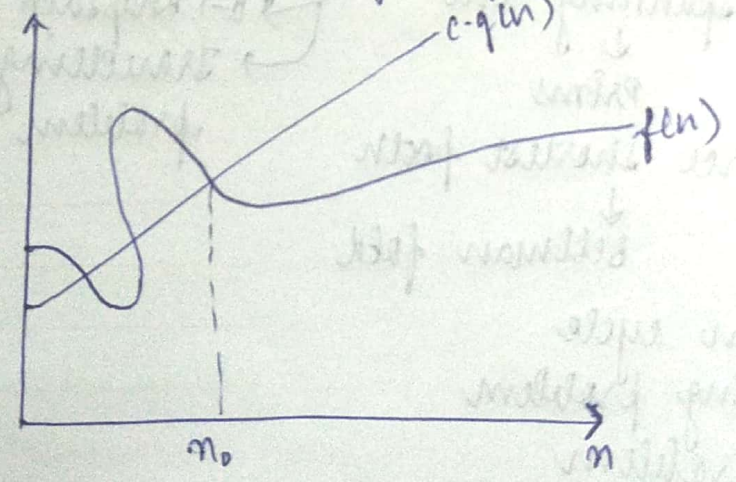
↳ Big-Oh (O) → Worst case

↳ Omega (Ω) → Best case

↳ Theta (Θ) → Average case

Big-Oh → indicates maximum time required by an algorithm

- It describes worst case of an algo. time comp.
- The function $f(n) = O(g(n))$ iff there exist a positive constants c & n_0 such that $f(n) \leq c \cdot g(n)$ for all n, $n \geq n_0$



Prove that :-

1) $\underbrace{3n+2}_{f(n)} = \underbrace{O(n)}_{g(n)}$

Let c=4

Scanned by CamScanner

n	3n+2		4.n
1	5	>	4
2	8	=	8
3	11	<	12
4			

Prove: $100n+6 = O(n)$

$c = 101$

1	106		101
2	206		202
3	306		303
4	406		404
5	506		505
6	606	=	606
7	706	<	707

Prove that :-

1) $3n+2 = \Omega(n)$

let $c=3$

n	3n+2		n
1	5	>	3
2	8		6
3	11		9
4	14		12

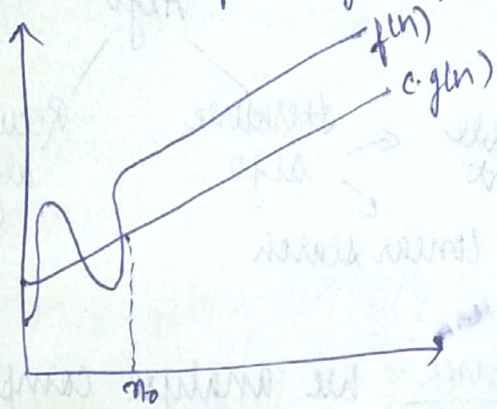
Average case :-

let $c_1=3$, $c_2=4$

n	$c_1 \cdot n$		3n+2		$c_2 \cdot n$
1	3		5		4
2	6		8		8
3	9	<	11	<	12
4					
5					

Omega(Ω) → It describes min. time required by an algo. for all I/P.

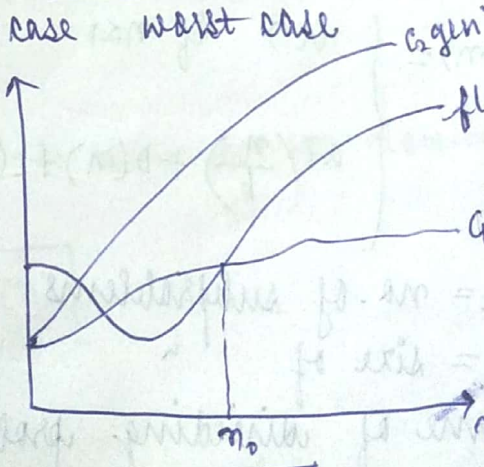
- It describes best case of algo. time complexity.
- $f(n) = \Omega g(n)$ iff there exists two constants c & n_0 such that $f(n) \geq c \cdot g(n)$ for all n, $n \geq n_0$.



Average case :-

Theta(θ) → $f(n) = \theta(g(n))$ iff there exists positive constants c_1, c_2, n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all n, $n \geq n_0$

Best case worst case



$f(n) = \theta g(n)$

Q:- Explain asymptotic notations with examples.

Q:- Prove

Scanned by CamScanner

Little Oh (o)

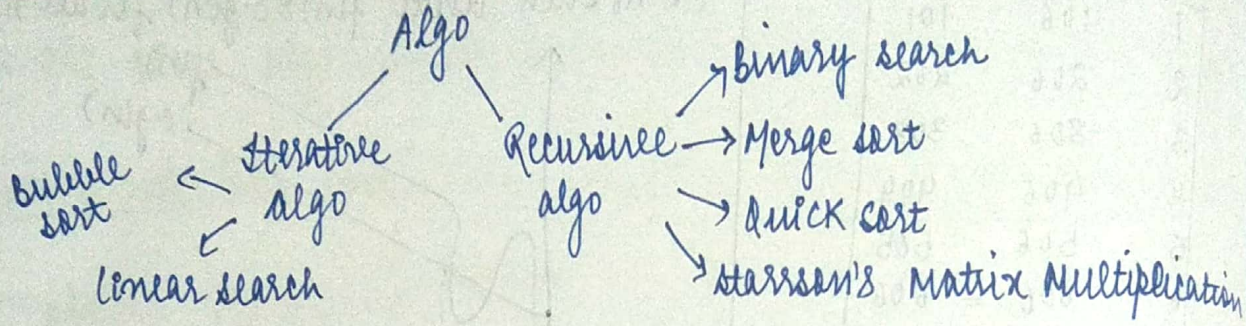
- $f(n) = o(g(n))$ iff

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

- $f(n) = \omega(g(n))$ iff

$$\lim_{n \to \infty} \frac{g(n)}{f(n)} = 0$$

ANALYSIS OF COMPLEXITY



Recurrence → we analyse complexity of recursive algo by recurrence function or eqn.

- A recurrence is an equation that describes the func. in terms of its values on smaller I/P.

For divide & conquer, recurrence eqn like

$$T(n) = \begin{cases} \theta(1) & \text{if } n \le 1 \\ aT\left(\frac{n}{b}\right) + D(n) + C(n), & \text{otherwise} \end{cases}$$

where a = no. of subproblems

where b = size of n

$D(n)$ = time of dividing problem into subproblems

$C(n)$ = time of combining the subproblem soln. to get the soln. for the problem of size n.

Binary search

1 2 3 4 5 6 7 (2)

$$T(n) = \begin{cases} \theta(1), & n \le 1 \\ 1 \cdot T\left(\frac{n}{2}\right) + O(1) + O(1) \end{cases}$$

Scanned by CamScanner

$T(n) = T\left(\frac{n}{2}\right) + 1$

$= \left[T\left(\frac{n}{4}\right) + 1\right] + 1$

$n \quad n^2 \log_2 n$

Merge sort :-

1 6 9 2 3 5

$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + O(1) + O(n) \end{cases}$

$T(n) = 2T\left(\frac{n}{2}\right) + n$

Methods to Solve Recurrence eqn :-

1) Master's theorem
2) Substitution method
3) Recursion tree method
4) Iterative method

Master's theorem :-

Let $f(n)$ be a function & $T(n)$ be defined on non-negative integers as :-

$$\boxed{T(n) = aT\left(\frac{n}{b}\right) + f(n)}$$

where a, b are constants.

Then, $T(n)$ can be bounded asymptotically as,

Case 1:

1) $T(n) = \theta(n^{\log_b a})$, if $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$
2) $T(n) = \theta(n^{\log_b a} \lg_2 n)$ if $f(n) = \theta(n^{\log_b a})$
3) $T(n) = \theta(f(n))$ if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon >$

Q :- Solve recurrence eqn :-

Scanned by CamScanner

$T(n) = 9T\left(\frac{n}{3}\right) + n$

$a = 9, \; b = 3, \; f(n) = n$

$n^{\log_b a} = n^{\log_3 9} = n^2 \Rightarrow n^{2-1}$

$f(n) = O(n^{2-1})$ for some $\epsilon > 0$

$$\boxed{T(n) = \theta(n^{\log_3 2}) = \theta(n^2)}$$

$T(n) = T\left(\frac{n}{2}\right) + 1$

$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$

$f(n) = \theta(n^{\log_2 1})$

$$\boxed{T(n) = \theta(\lg n)}$$

$T(n) = 2T\left(\frac{n}{3}\right) + 1$

$a = 2, \; b = 3$

$n^{\log_b a} = n^{\log_3 2}$

$T(n) = T\left(\frac{n}{2}\right) + n^2$

$a = 1, \; b = 2, \; f(n) = n^2$

$n^{\log_b a} = n^{\log_2 1} = n^0 \Rightarrow n^2$

$f(n) = \theta(n^{\log_2 1})$

$$\boxed{T(n) = \theta(n^2)}$$

$T(n) = 2T\left(\frac{n}{2}\right) + n$

$a = 2, \; b = 2$

$n^{\log_b a} = n^{\log_2 2} = n$

$$\boxed{T(n) = n \lg_{10} n}$$

Scanned by CamScanner

$T(n) = 4T\left(\frac{n}{2}\right) + n$

$a = 4, b = 2$

$n^{\log_b a} = n^{\log_2 4} = n^{\log_2 (2)^2} = n^{2\log_2 2}$

$= n^{2-1}$

$= n^{2-1} = n$

$T(n) = \theta(n \log_b a)$

$\boxed{T(n) = \theta(n^2)}$

$T(n) = 4T\left(\frac{n}{2}\right) + n^2$

$a = 4, b = 2$

$n^{\log_b a} = n^{\log_2 4} = n^2$

$\boxed{T(n) = n^2 \log n}$

$T(n) = 4T\left(\frac{n}{2}\right) + n^3$

$n^{\log_b a} = n^{\log_2 4} = n^{2+1} = n^3$

$\boxed{T(n) = \theta(n^3)}$

$T(n) = T\left(\frac{2n}{3}\right) + 1$

$a = 1, b = 3/2$

$n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$

$\boxed{T(n) = \log_{10} n}$

$T(n) = T(\sqrt{n}) + 1 = T(n^{1/2}) + 1$

$a = 1, b = 1$

$n^{\log_b a} = n^{\log_1 1} = n^0 = 1$

$T(n) = \theta 1$

Scanned by CamScanner

Merge Sort

Algorithm :-

Merge sort (A, p, r)

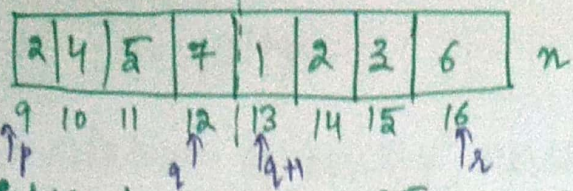
1) if p < r
2) then q ← ⌊(p+r)/2⌋ (mid, floor)
3) Merge sort (A, p, q)
4) Merge sort (A, q+1, r)
5) Merge (A, p, q, r)

Merge (A, p, q, r)

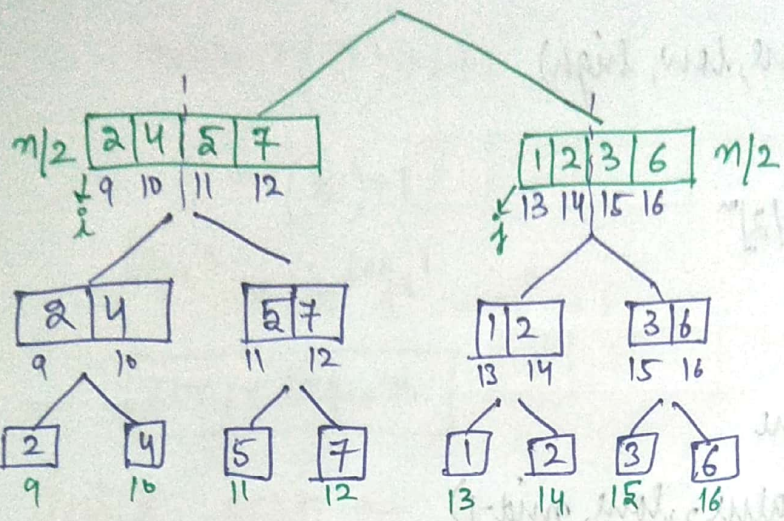
1) $n_1 \leftarrow q-p+1$
2) $n_2 \leftarrow r-q$
3) create arrays L[1 — n_1+1] & R[1 — n_2+1]
4) for i ← 1 to n_1
5) do L[i] ← A[p+i-1]
6) for j ← 1 to n_2
7) do R[j] ← A[q+j]
8) L[n_1+1] ← ∞
9) R[n_2+1] ← ∞
10) i ← 1
11) j ← 1
12) for k ← p to r
13) do if L[i] ≤ R[j]
14) then A[k] ← L[i]
15) i ← i+1
16) else A[k] ← R[j]
17) j ← j+1

Scanned by CamScanner

Ex



$q = \left\lfloor \frac{p+r}{2} \right\rfloor = \frac{9+16}{2} = \frac{25}{2} = 12.5$



1) $n_1 \leftarrow 12 - 9 + 1 = 4$

2) $n_2 \leftarrow 16 - 12 = 4$

L = | 2 | 4 | 5 | 7 | ∞ |

Q = | 1 | 2 | 3 | 6 | ∞ |

A = | 1 | 2 | 2 | 3 | 4 | 5 | 6 | 7 |

Analysis of Complexity :-

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$

$T(n) = 2T\left(\frac{n}{2}\right) + D(n) + C(n)$

$T(n) = 2T\left(\frac{n}{2}\right) + C + n$

$$\boxed{T(n) = 2T\left(\frac{n}{2}\right) + n}$$

$n^{\log_b a} = n^{\log_2 2} = n$ → using Master's

$$\boxed{T(n) = n \log_{10} n}$$

Scanned by CamScanner

BINARY SEARCH

Algorithm :-

```
low = 0
high = n-1
Binary search (A, value, low, high)
 if (low <= high)
 {
 mid = ⌊(low+high)/2⌋
 if (A[mid] == value)
  return mid,
else if (A[mid] > value
 Binary search (A, value, low, mid-1)
else
 Binary search (A, value, mid+1, high)
 }
 return not found;
}
```

2	4	5	7	8	9	12	14	17	19	22	25	27	28	33
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

22

low	mid	high
0	7	14
8	11	14
8	9	10
10	10	10

found at 10

34

low	mid	high
0	7	14
8	11	14
12	13	14
14	14	14

Not found

2

low	mid	high
0	7	14
0	3	7
0	1	2
0	0	0

found at 0

Scanned by CamScanner

Analysis of Complexity:-

Recursive relation of binary search:-

General:- $T(n) = aT\left(\frac{n}{b}\right) + D(n) + C(n)$

$$T(n) = T\left(\frac{n}{2}\right) + 1 + 1$$

$$= T\left(\frac{n}{2}\right) + 1$$

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$$

$$\boxed{T(n) = \log_{10} n}$$

Quick Sort :-

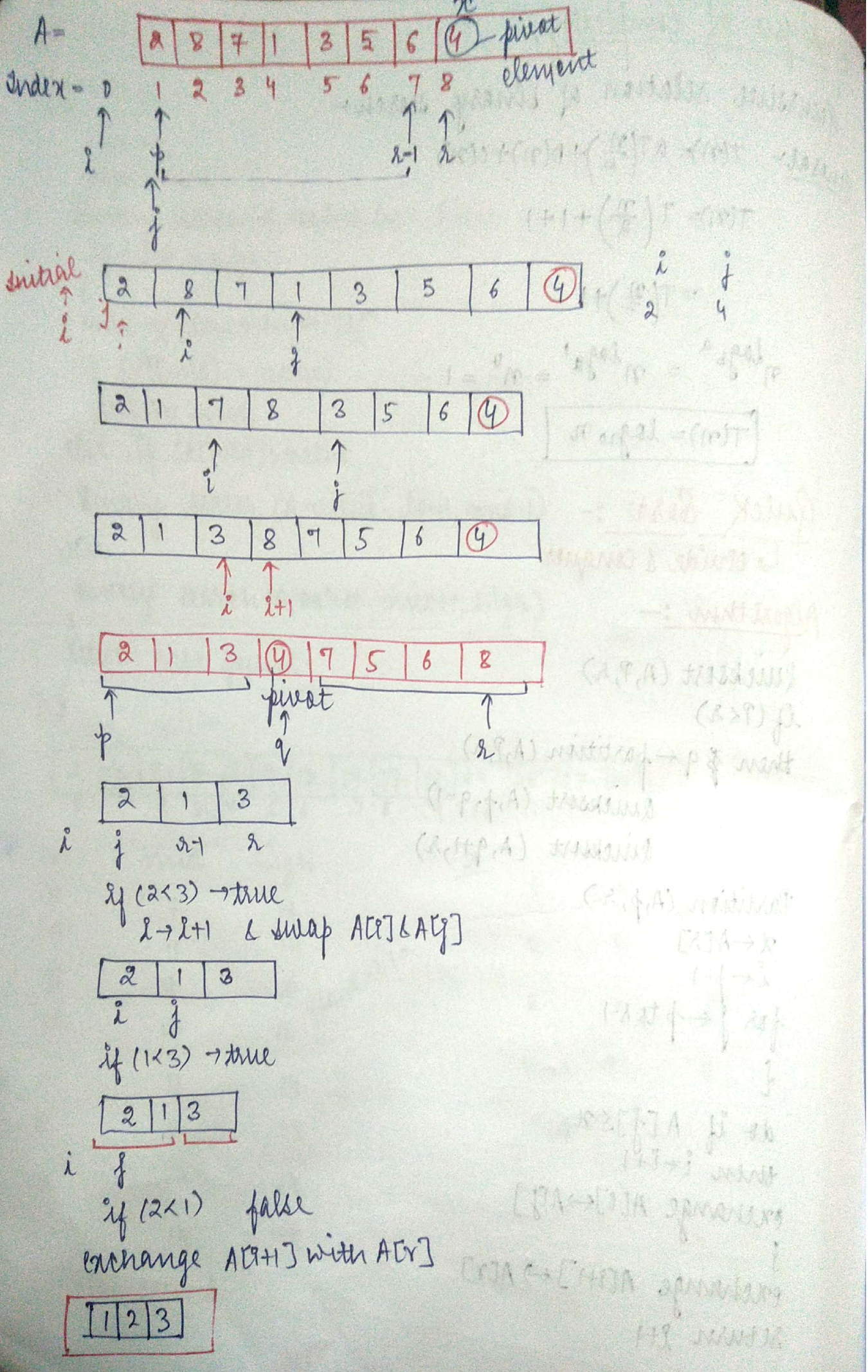
↳ Divide & conquer

Algorithm :-

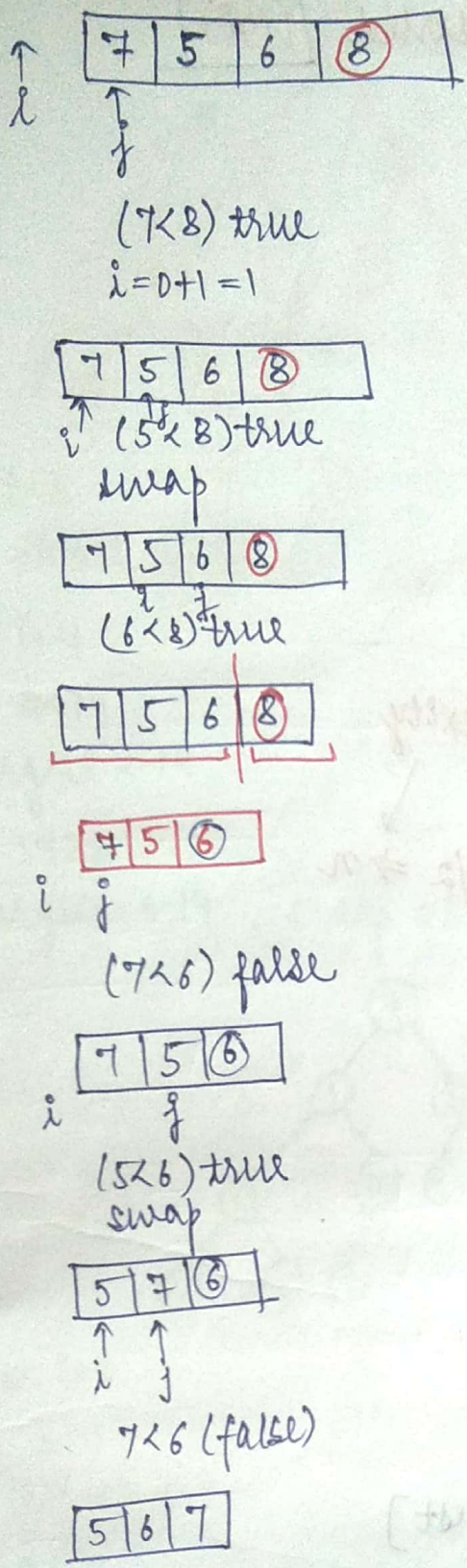
```
Quicksort (A, P, r)
if (P < r)
then q ← partition (A, P, r)
        Quicksort (A, p, q-1)
        Quicksort (A, q+1, r)

Partition (A, p, r)
 x ← A[r]
 i ← p-1
for j ← p to r-1
{
do if A[j] ≤ x
then i ← i+1
exchange A[i] ↔ A[j]
}
exchange A[i+1] ↔ A[r]
return i+1
```

Scanned by CamScanner



Scanned by CamScanner



Complexity of Quick Sort

Best & Average case:- $O(n \log n)$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1) + n$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Scanned by CamScanner

Worst case:- when list is reverse sorted. $O(n^2)$

8 7 6 5 4 3 2 | 1

$n-1$

$T(n) = T(n-1) + 1 + n$

$T(n) = T(n-1) + n$

Heap Sort :-

Algo:

Build-Max Heap (A)

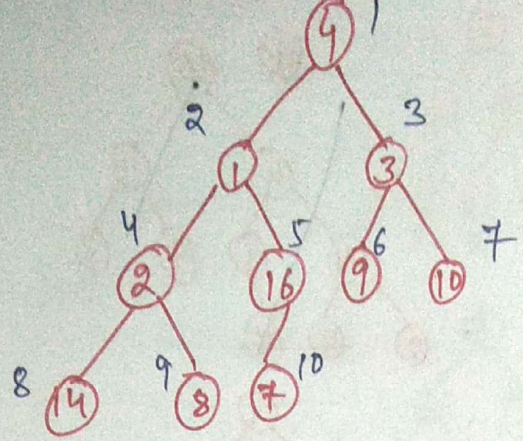
1) heapsize[A] ← length[A] → 1 (complexity)
2) for i ← [length[A]/2] down to 1 → $n/2 \Rightarrow n$
3) do Max-heapify (A, i) → $\log n$

Max-heapify(A, i)

1) l ← LEFT [i]
2) r ← RIGHT (i)
3) if l ≤ heapsize [A] & A[l] > A[i]
4) then largest ← l
5) else largest ← i
6) if r ≤ heapsize [A] & A[r] > A[largest]
7) then largest ← r
8) if largest ≠ i
9) then exchange (A[i] ↔ A[largest])
10) Max-heapify (A, largest)

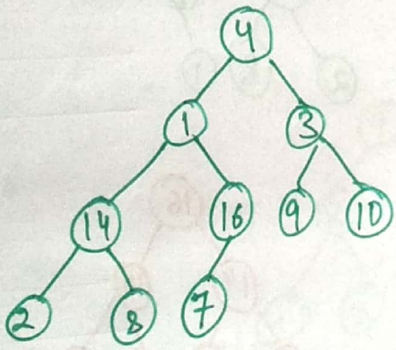
Ex→ 4, 1, 3, 2, 16, 9, 10, 14, 8, 7

Scanned by CamScanner

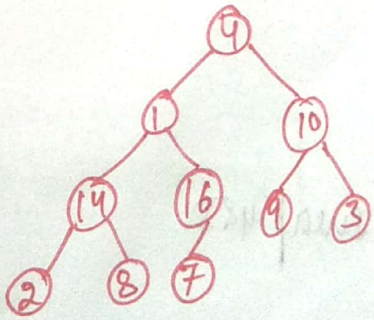


for i=5
same tree.

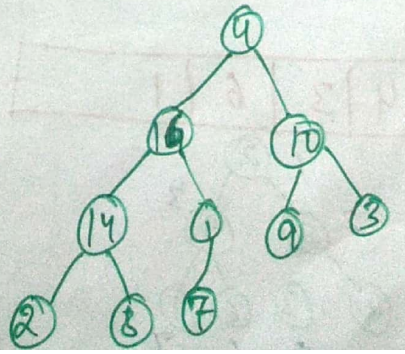
for i=4
2<14
largest ← 14
14>8
largest ← 14 , swap 2 & 14



for i=3
3<9
largest ← 9
9<10
largest ← 10, swap 3 & 10



for i=2
1<2 14
largest ← 2 14
14 2 < 16
largest ← 16 , swap 2 & 16



Scanned by CamScanner

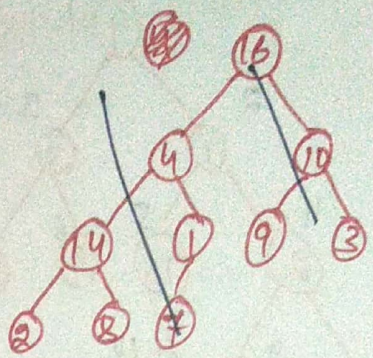
for i=1

4<16

largest ← 16

16>10

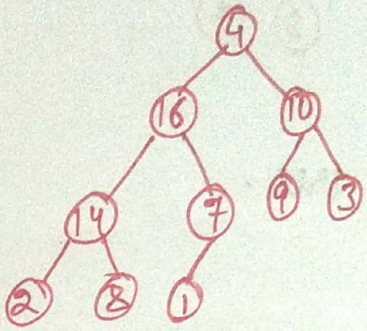
largest ← 16, swap 16 & 4



when i=2.

1<7

largest ← 7, swap 1 & 7



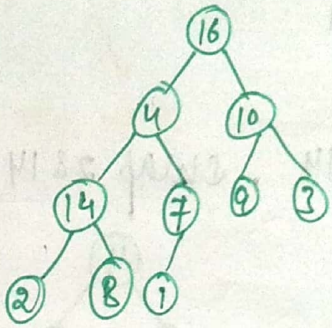
when i=1

4<16

largest ← 16

16>10

largest ← 16, swap 16 & 4

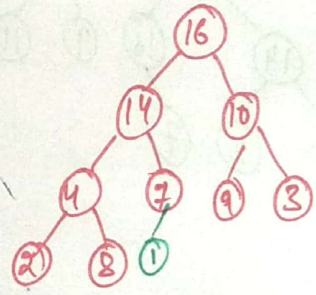


4<14

largest ← 14

14>7

largest ← 14, swap 4 & 14

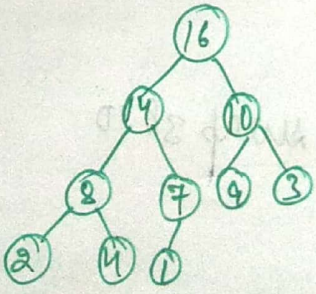


4<2

largest ← 4

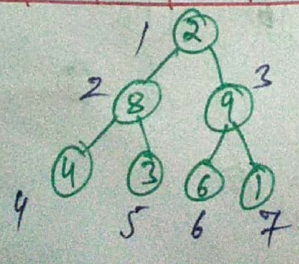
2<8

largest ← 8, swap 4 & 8



⇒

2	8	9	4	3	6	1

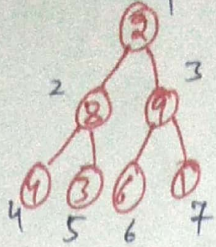


Scanned by CamScanner

when i=3

9 > 6

6 > 1



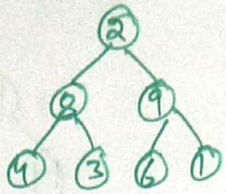
when i=2.

8 < 4

l ← 8

4 > 3

l ← 8



when i=1

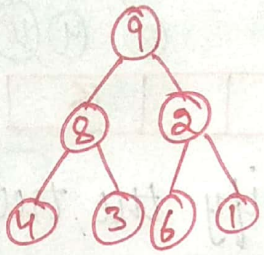
2 < 8

largest ← 8

8 < 9

largest ← 9

swap 2 & 9



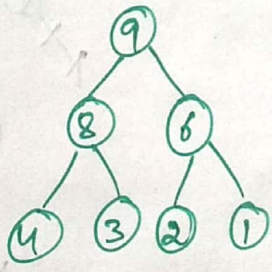
2 < 6

largest ← 6

6 > 1

largest ← 6

swap 2 & 6



Height of tree = log n

Complexity of Build Max heap = O(n log n)

Heap Sort (A)

1) Build-Max heap (A)

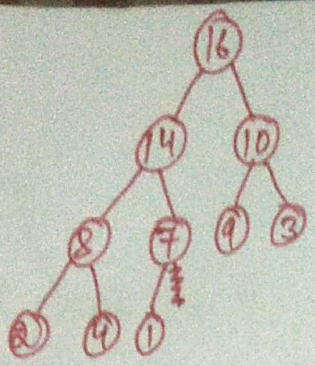
2) for i ← length[A] down to 2

3) do exchange A[1] ↔ A[i]

4) heap-size[A] = heap-size[A] - 1

5) Max-heapify (A, 1)

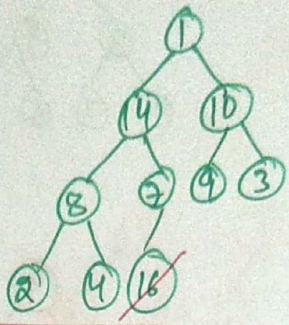
Scanned by CamScanner



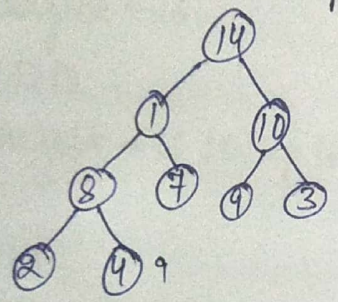
for $i \leftarrow 10$ to 2.

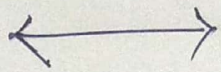
$A[1] \leftrightarrow A[10]$

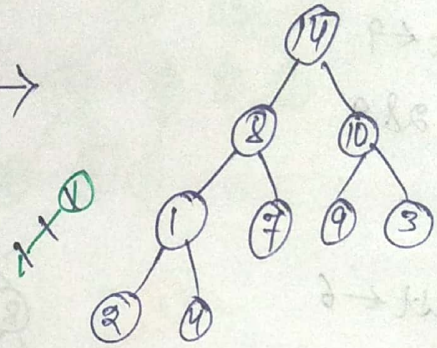
$\therefore$ Resulting tree



Now, call max heap & heapify the tree



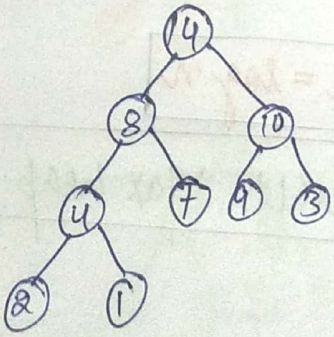




for $i=9$

$A[1] \leftrightarrow A[9]$

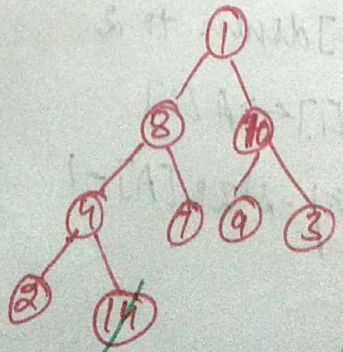
$\therefore$ Resulting tree

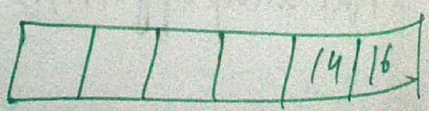


for $i=9$

$A[1] \leftrightarrow A[9]$

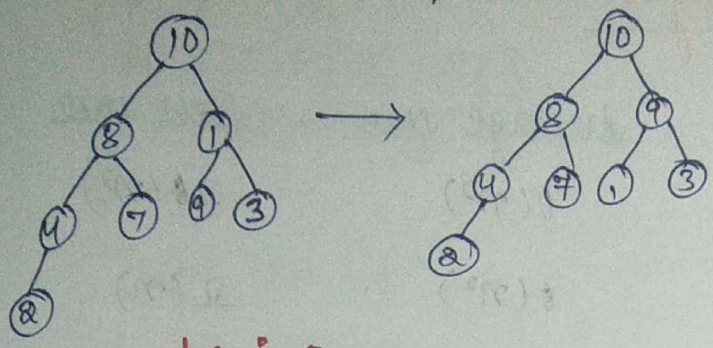
$\therefore$ Resulting tree



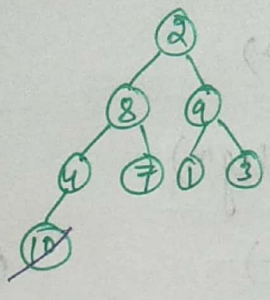


Scanned by CamScanner

Now, call max heap (heapify tree

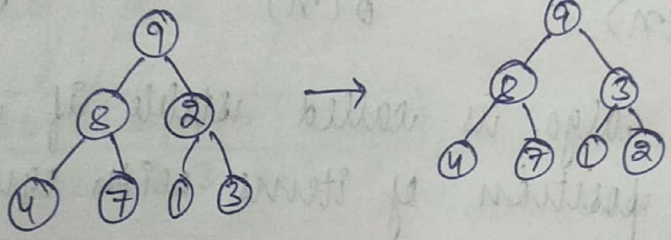


for i=8
A[1] ↔ A[8]
∴ Resulting tree

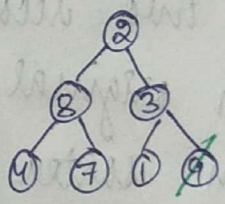


			10	14	16

Call max heap.

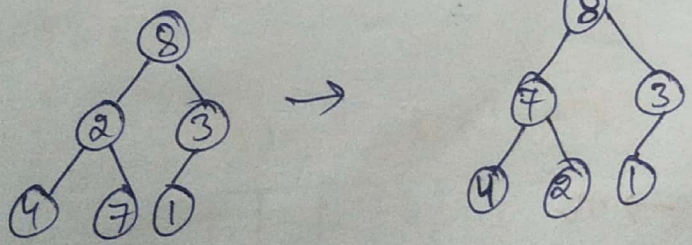


for i=7
A[1] ↔ A[7]
∴ Resulting tree



		9	10	14	16

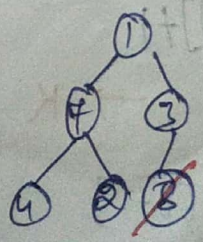
Call max heap



Final:

1	2	3	4	7	8	9	10	14	16

for i=6
A[1] ↔ A[6]
∴ Resulting tree



	8	9	10	14	16

Scanned by CamScanner

Complexity of Sorting algo:-

Sorting	worst case	Average case	Best case
1) Bubble sort	$O(n^2)$	$\theta(n^2)$	$\Omega(n^2)$
2) Insertion sort	$O(n^2)$	$\theta(n^2)$	$\Omega(n)$
3) Selection sort	$O(n^2)$	$\theta(n^2)$	$\Omega(n^2)$
4) Merge sort	$O(n \log n)$	$\theta(n \log n)$	$\Omega(n \log n)$
5) Quick sort	$O(n^2)$	$\theta(n \log n)$	$\Omega(n \log n)$
6) Heap sort	$O(n \log n)$	$\theta(n \log n)$	$\Omega(n \log n)$
7) Counting sort	$O(n)$	$\theta(n)$	$\Omega(n)$

Stable → A sorting algo. is called stable if it does not exchange relative position of items with equal keys. Suppose, R & S are two records with same key & R appears before S in original list, then R will always appear before S in sorted list.

Counting Sort :-

ALGORITHM:

```
for i ← 0 to K          → K
do C[i] ← 0
for j ← 1 to length[A]  → n
do C[A[j]] ← C[A[j]] + 1
for i ← 1 to K          → K
do C[i] ← C[i] + C[i-1]
```

Scanned by CamScanner

```
for j ← length[A] down to 1 → n
{
  do B[C[A[j]]] ← A[j]
     C[A[j]] ← C[A[j]] - 1
}
```

Total Complexity :-

$= K + n + K + n$

$= 2n$

$= O(n)$

Ex →

1	2	3	4	5	6	7	8
3	6	4	1	3	4	1	4

Max = 6 = K → New array of length 6

C =

0	1	2	3	4	5	6
0	2	0	2	3	0	1

C =

0	1	2	3	4	5	6
0	2	2	4	7	7	8

A =

1	2	3	4	5	6	7	8
3	6	4	1	3	4	1	4

j → 8

j → 8 to 1

for j = 8
A[j] = 4
C[4] = 7
Copy A[j] to B[7] . . .
& decrease C[4] by 1

B =

1	2	3	4	5	6	7	8
1	1	3	3	4	4	4	6

C =

0	1	2	3	4	5	6
0	2	2	3	4	5	6

for j = 8

A[8] = 4 , C[4] = 7 , B[7] = A[8]

B =

						4	

C =

0	1	2	3	4	5	6
0	2	2	4	6	7	8

Scanned by CamScanner

for j=7

A[7]=1 , C[1]=2 , B[2]=A[7]=1

B=

	1					4	
1	2	3	4	5	6	7	8

C=

0	1	2	4	6	7	8
0	1	2	3	4	5	6

for j=6

A[6]=4 , C[4]=6 , B[6]=A[6]=4

B=

	1				4	4	
1	2	3	4	5	6	7	8

C=

0	1	2	4	5	7	8
0	1	2	3	4	5	6

for j=5

A[5]=3, C[3]=4 , B[4]=A[5]=3

B=

	1		3		4	4	
1	2	3	4	5	6	7	8

C=

0	1	2	3	5	7	8
0	1	2	3	4	5	6

for j=4

A[4]=1 , C[1]=1 , B[1]=A[4]=1

B=

1	1		3		4	4	
1	2	3	4	5	6	7	8

C=

0	0	2	3	5	7	8
0	1	2	3	4	5	6

for j=3

A[3]=4, C[4]=5, B[5]=A[3]=4

B=

1	1		3	4	4	4	
1	2	3	4	5	6	7	8

C=

0	0	2	3	4	7	8
0	1	2	3	4	5	6

Scanned by CamScanner

for j=2

A[2]=6 , C[6]=8 , B[8]=A[2]=6

B= | | | | 3 | 4 | 4 | 4 | 6 |
1 2 3 4 5 6 7 8

C= | 0 | 0 | 2 | 3 | 4 | 4 | 7 |
0 1 2 3 4 5 6

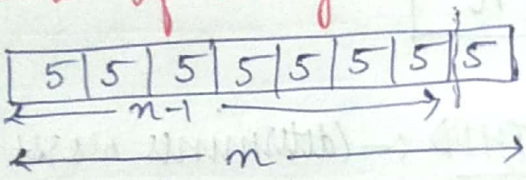
for j=1

A[1]=3 , C[3]=3 , B[3]=A[1]=3

B= | 1 | 1 | 3 | 3 | 4 | 4 | 4 | 6 |
1 2 3 4 5 6 7 8

C= | 0 | 0 | 2 | 2 | 4 | 7 | 7 |
0 1 2 3 4 5 6

Q:- What is running time of quick sort when all the elements of array A has same value.



$O(n^2)$

Solve using Master's Method:-

$T(n) = T(\sqrt{n}) + 1$

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$ ——①

$T(n) = T(n^{1/2}) + 1$

By changing the variable method :-

Suppose, $n = \log_2 n$

$n = 2^m$

Put the value of n in eq ①

$T(2^m) = T(2^{m/2}) + 1$

$S(m) = S(m/2) + 1$

$f(m) = 1$

$m^{\log_b a} = m^{\log_2 1}$

$= m^0 = 1 = f(n)$

case 2 holds :-

$T(n) = m^{\log_b a} \lg m$

$= \lg m$

$= O(\lg \lg_2 n)$

Scanned by CamScanner

$T(n) = 2T(\sqrt{n}) + n$

let $m = \log_2 n$

$n = 2^m$

$T(2^m) = 2T(2^{m/2}) + 2T(\sqrt{2^m}) + 2^m$

$S(m) = 2S(m/2) + 8m$

$a = 2,\ b = 2,\ f(m) = m$

$m^{\log_b a} = m^{\log_2 2} = m$

Case 2 holds:-

$T(n) = n^{\log_b a} f(m) \lg m$

$= m \lg m$

$\boxed{T(n) = \log_2 n \lg \log_2 n}$

SUBSTITUTION METHOD :- (determines worst case complexity)

① guess a solution

② Prove by induction

Q:- $T(n) = 2T(\frac{n}{2}) + n$

If $f(n) = n$, assume $n \log n$

If $f(n) = 1$, assume $\log n$

$T(n) = O(2T(\frac{n}{2}) + n)$

$T(n) \leq 2T(\frac{n}{2}) + n$ ——— ①

Assume, $T(n) = O(n \log n)$

$T(n) \leq n \log n$

Putting $n = n/2$ in eq ① Resulting eq ①

$T(n/2) \leq 2 \cdot \frac{n}{2} \log \frac{n}{2} + n$ $T(n) \leq 2 \cdot \frac{n}{2} \log \frac{n}{2} + n$

Scanned by CamScanner

$T(n) \leq n\log n - n\log 2 + n$

$\leq n\log n - n + n$

$\boxed{T(n) \leq n\log n}$

Hence proved

Q:- $T(n) = 2T\left(\left\lfloor\frac{n}{2}\right\rfloor + 17\right) + n$

$T(n) \leq 2T\left(\frac{n}{2} + 17\right) + n$ — ①

Assume, $T(n) = O(n\log n)$

$T(n) \leq n\log n$

$T(n) \leq 2 \cdot \frac{n}{2}\log\frac{n}{2} + 34 + n$

$T(n) \leq n\log n - n\log 2 + n$

$\leq n\log n - n + n$

$\boxed{T(n) \leq n\log n}$

ITERATIVE METHOD :-

Q:- find running time/complexity of the following:-

$T(n) = 3T\left(\left\lfloor\frac{n}{4}\right\rfloor\right) + n$ — ①

$T\left(\frac{n}{4}\right) = 3T\left(\frac{n}{4\cdot 4}\right) + \frac{n}{4}$

Putting the value in eq ①

$= 3\left[3T\left[\frac{n}{4^2}\right] + \frac{n}{4}\right] + n$

$= 3^2 T\left[\frac{n}{4^2}\right] + \frac{3n}{4} + n$

$= 3^2\left[3\cdot T\left[\frac{n}{4^3}\right] + \frac{n}{4^2}\right] + \frac{3}{4}n + n$ [By putting value of $T\left(\frac{n}{4^2}\right)$]

Scanned by CamScanner

$$= 3^3 T\left[\frac{n}{4^3}\right] + \left(\frac{3}{4}\right)^2 n + \frac{3}{4} n + n$$

$$= 3^3\left[3 \cdot T\left[\frac{n}{4^4}\right] + \frac{n}{4^3}\right] + \frac{3}{4} n + n$$

$$= 3^4 T\left[\frac{n}{4^4}\right] + \left(\frac{3}{4}\right)^3 n + \left(\frac{3}{4}\right)^2 n + \left(\frac{3}{4}\right) n + n$$

$$= 3^K T\left[\frac{n}{4^K}\right] + \underbrace{\left(\frac{3}{4}\right)^{K-1} n + \cdots \left(\frac{3}{4}\right)^1 n + \left(\frac{3}{4}\right)^0 n}$$

I II

Suppose $T(1) = 1$

$$T\left(\frac{n}{4^K}\right) = 1$$

$$\frac{n}{4^K} = 1$$

$$n = 4^K$$

$$\boxed{K = \log_4 n}$$

putting value of K in I

$$= 3^{\log_4 n}(1)$$

$$= n^{\log_4 3}$$ (base change)

$$= n^{<1}$$

II $\quad a^0 + a^1 + a^2 + \cdots a^{K-1} = \frac{a^0[1 - a^K]}{1 - a}$

$$= \left(\frac{3}{4}\right)^0 n + \left(\frac{3}{4}\right)^1 n + \left(\frac{3}{4}\right)^2 n + \cdots \left(\frac{3}{4}\right)^{K-1} n$$

$$= n\left[\left(\frac{3}{4}\right)^0 + \left(\frac{3}{4}\right)^1 + \left(\frac{3}{4}\right)^2 + \cdots \left(\frac{3}{4}\right)^{K-1}\right]$$

Scanned by CamScanner

$$= n\left[\frac{\left(\frac{3}{4}\right)^0\left[1-\left(\frac{3}{4}\right)^K\right]}{1-\frac{3}{4}}\right]$$

$$= 4n\left[1-\left(\frac{3}{4}\right)^{\log_4 n}\right] = 4n\left[1-n^{\log_4 3/4 < 1}\right]$$

$$= 4n$$

Neglecting values < 1

Finally, $n^{<1} + 4n = n$

$\therefore$ **Complexity $= O(n)$**

Q: $T(n) = 8T\left(\frac{n}{2}\right) + n^2$

$$= 8\left[8T\left(\frac{n}{2^2}\right) + \left(\frac{n}{2}\right)^2\right] + n^2$$

$$= 8^2 T\left(\frac{n}{2^2}\right) + 8\left(\frac{n}{2}\right)^2 + n^2$$

$$= 8^2\left[8T\left(\frac{n}{2^3}\right) + \left(\frac{n}{2^2}\right)^2\right] + n^2 + 8\left(\frac{n}{2}\right)^2$$

$$= 8^3 T\left(\frac{n}{2^3}\right) + 8^2\left(\frac{n}{2^2}\right)^2 + 8\left(\frac{n}{2}\right)^2 + n^2$$

$$= 8^K T\left[\frac{n}{2^K}\right] + (8)^{K-1}$$

$$= 8^3 T\left(\frac{n}{2^3}\right) + 4n^2 + 2n^2 + n^2$$

$$= 8^K T\left[\frac{n}{2^K}\right] + (2)^0 n^2 + (2)n^2 + (2)^2 n^2 + \cdots (2)^{K-1} n^2$$

Scanned by CamScanner

If value of $a>1$ then $\frac{1}{1-a}$

if $a<1$ then $\frac{a^0[1-a^K]}{[1-a]}$

I $T(1)=1$

$T\left(\frac{n}{2^K}\right)=1$

$n=2^K$

$\boxed{K=\log_2 n}$

putting K in I

$=8^{\log_2 n}(1)$

$=n^{\log_2 8}$

$\boxed{=n^3}$

II $\frac{1}{1-a}=\frac{1}{1-2}=-1(n^2)=-n^2$

Soln: $I+II=n^3-n^2$

$=n^3$

$\therefore$ $\boxed{\text{Complexity} = O(n^3)}$

Q. $T(n)=2T\left(\frac{n}{2}\right)+n$

$=2\left[2T\left(\frac{n}{2^2}\right)+\frac{n}{2}\right]+n$

$=2^2T\left(\frac{n}{2^2}\right)+n+n$

$=2^2\left[2T\left(\frac{n}{2^3}\right)+\frac{n}{2^2}\right]+n+n$

$=2^3T\left(\frac{n}{2^3}\right)+n+n+n$

$=2^3\left[2T\left(\frac{n}{2^4}\right)+\frac{n}{2^3}\right]+3n$

Scanned by CamScanner

$= 2^4 T\left(\frac{n}{2^4}\right) + 4n$

$= 2^k T\left(\frac{n}{2^k}\right) + kn$

I II

(I) $T(1) = 1$

$T\left(\frac{n}{2^k}\right) = 1$

$\boxed{k = \log_2 n}$

putting value of k in (I)

$= 2^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right)$

$= n^{\log_2 2} T\left(\frac{n}{n^{\log_2 2}}\right)$

$= n$

I + II ⇒

$T(n) = n + k(n) = n + n\log_2 n$

$= n(1+k)$

now, $n\log n > n$

∴ $\boxed{\text{complexity} = O(n\log n)}$

Q. $T(n) = T(n-1) + n$

$= T(n-2) + (n-1) + n$

$= T(n-3) + (n-2) + (n-1) + n$

$= T(n-4) + (n-3) + (n-2) + (n-1) + n$

$= \underbrace{T(n-k)}_{\text{I}} + \underbrace{\text{---}(n-1) + n}_{\text{II}}$

Scanned by CamScanner

for (I) :-

$T(1) = 1$

$T(n+K) = 1$

$\boxed{K = n-1}$

putting value of K in (I)

$= T(n-(n-1))$

$= T(1)$

$= 1$

for (II) :-

$= n + (n-1) + (n-2) + \cdots 1 + 0$

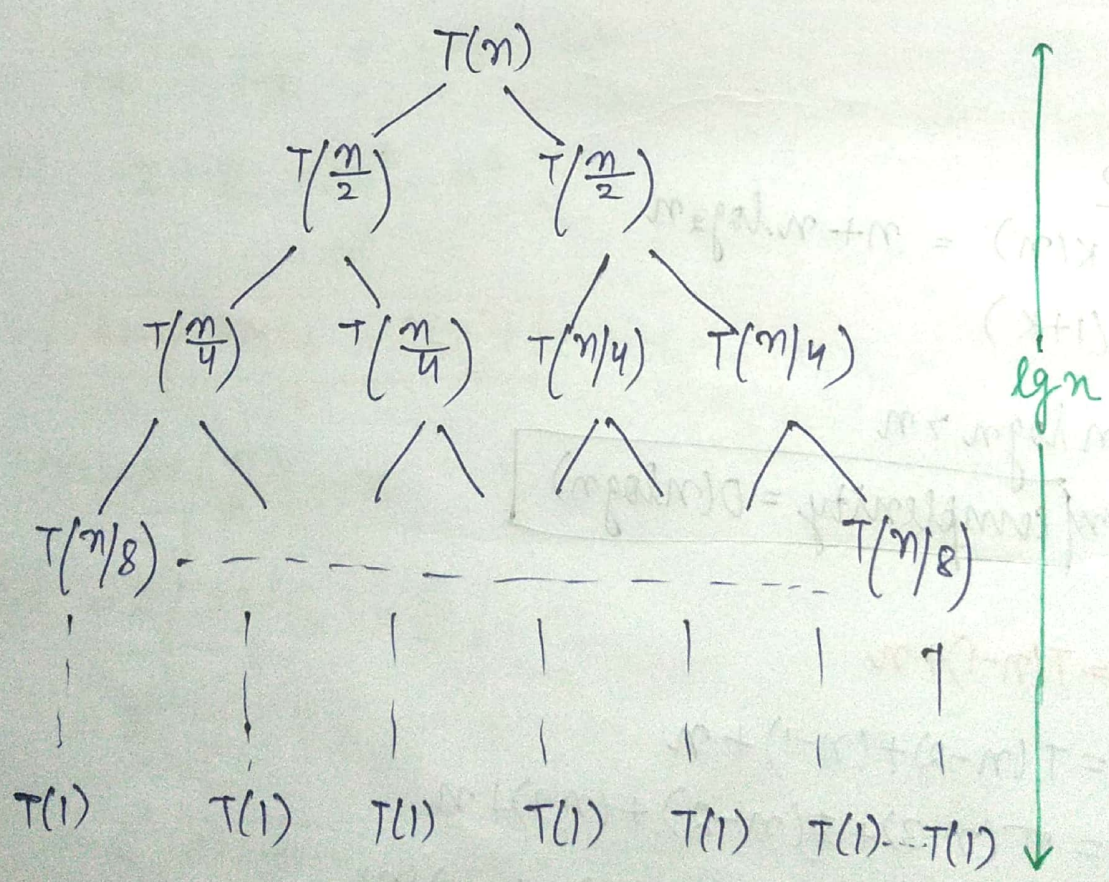
$= \frac{n(n+1)}{2}$

$= \frac{n^2+n}{2}$

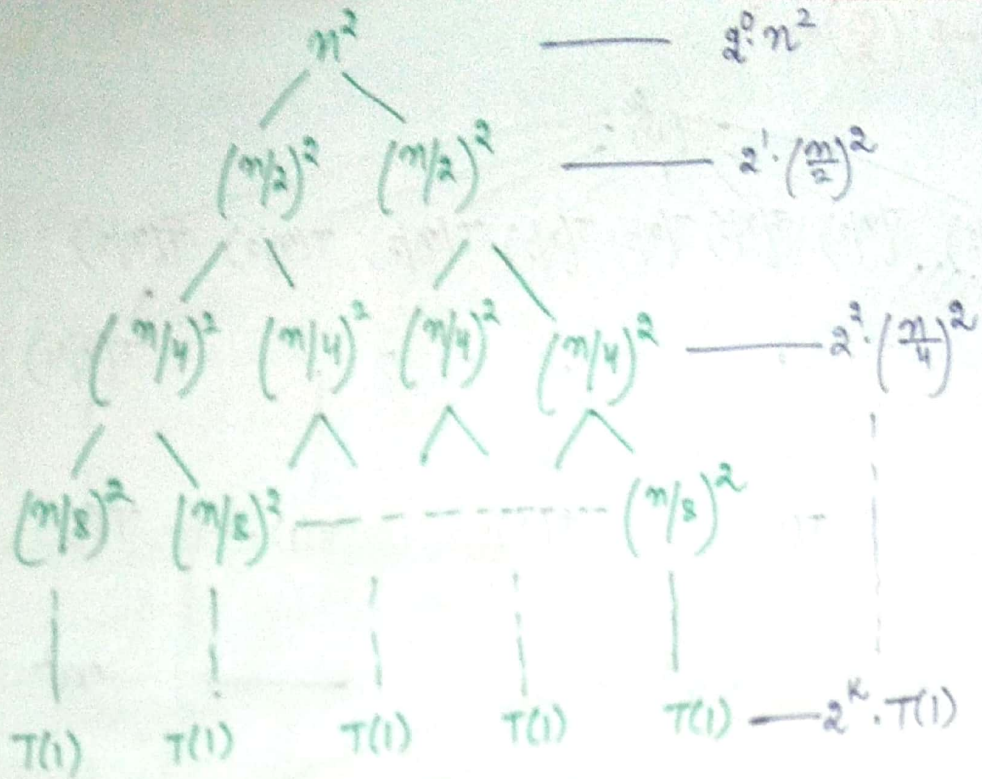
Total complexity = (I) + (II) $= \boxed{n^2}$

RECURSION TREE METHOD :-

$T(n) = 2T\left(\frac{n}{2}\right) + n^2$



Scanned by CamScanner



Complexity at last level = no. of nodes at last level * T(1)

let $T(1) = 1$

$T\left(\frac{n}{2^k}\right) = 1$

$\frac{n}{2^k} = 1$

$n = 2^k$

$\Rightarrow \boxed{k = \log_2 n}$

putting value of k in last level

$= 2^{\log_2 n} \cdot (1)$

$= n^{\log_2 2}$

$= n$

Total complexity

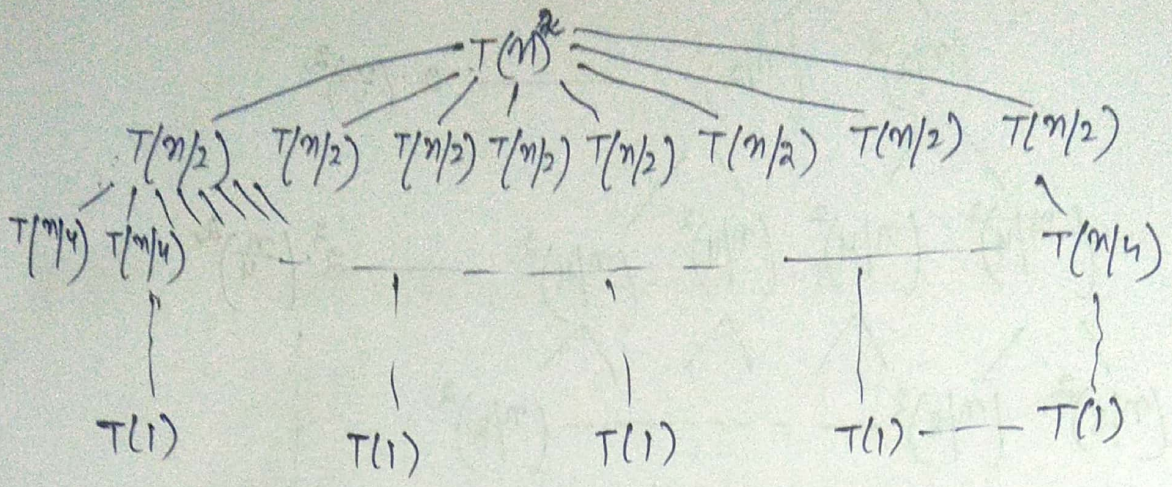
$= (n^2 + n^2/2 + n^2/4 + n^2/8 + \cdots) +$ complexity at last level

$= n^2 \left[1 + 1/2 + 1/4 + \cdots\right] + n$

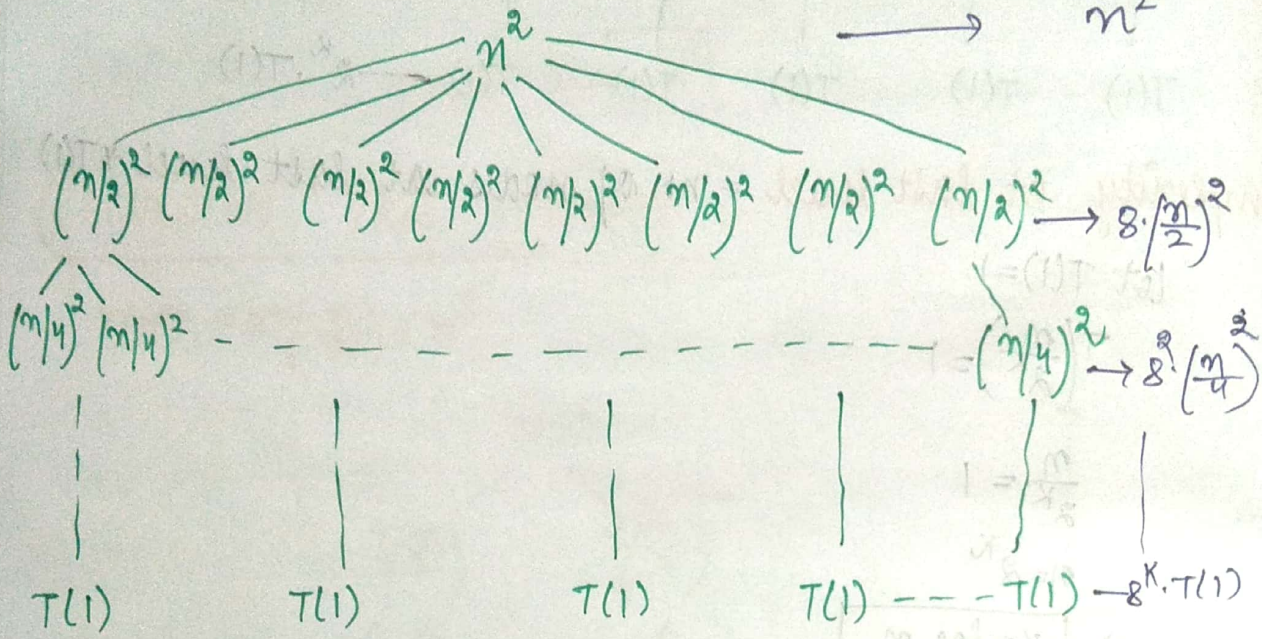
$= n^2 \left[\frac{1}{1-1/2}\right] + n \quad = n^2 + n \quad \boxed{= O(n^2)}$

Scanned by CamScanner

$T(n) = 8T\left(\frac{n}{2}\right) + n^2$



$\longrightarrow n^2$



Complexity at last level = no. of nodes at last level * T(1)

let $T(1) = 1$

$T\left(\frac{n}{2^K}\right) = 1$

$\frac{n}{2^K} = 1$

$n = 2^K$

$\boxed{K = \log_2 n}$

putting K in last level $= 8^{\log_2 n}$

$= n^{\log_2 8}$

$= n^3$

Scanned by CamScanner

Total complexity

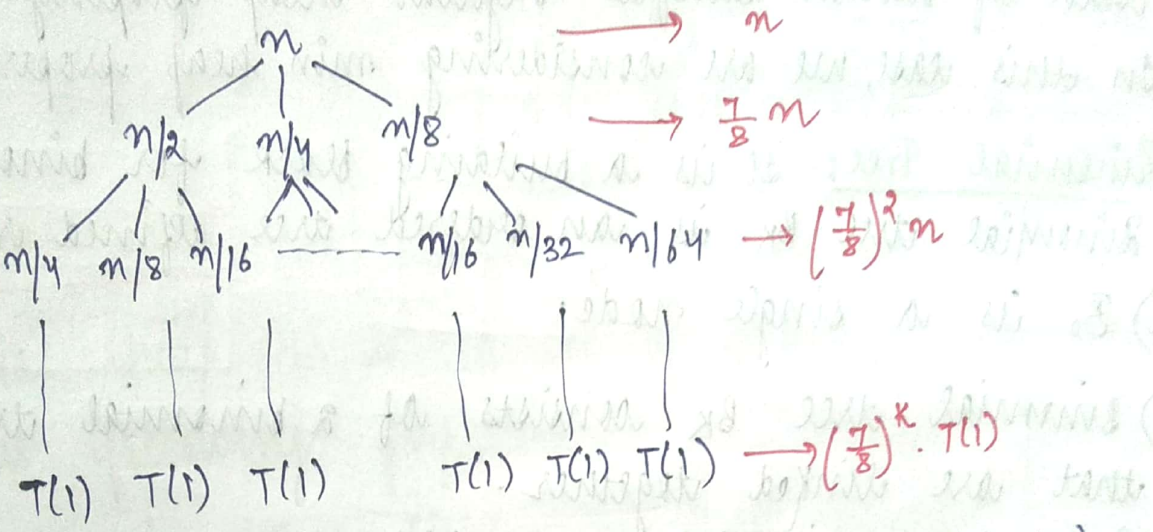
$= (n^2 + 2n^2/2 + 4n^2/4 + \cdots) +$ Complexity at last level (n^3)

$= n^2[1+2+4+\cdots] + n^3$

$= n^2\left[\frac{1}{1-2}\right] + n^3$

$\boxed{= n^3}$

Q:- $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + n$



Complexity at last level = no. of nodes at last level * T(1)

Let $T(1) = 1$

$T\left(\frac{n}{8^K}\right) = 1$

$\left(\frac{n}{8^K}\right) = 1$

$n = 8^K$

$\boxed{K = \log_8 n}$

putting K in last level

$= \left(\frac{7}{8}\right)^{\log_8 n} \cdot 1 = n^{\log_8 (7/8)}$

Total complexity $= n + \frac{7}{8}n + \left(\frac{7}{8}\right)^2 n + \cdots + n^{\log_8 (7/8)}$

$= n\left[1 + \frac{7}{8} + \left(\frac{7}{8}\right)^2 + \cdots\right] + n^{\log_8 (7/8)}$

Scanned by CamScanner

$= n\left[1 - \frac{1}{7/8}\right] + n^{\log_3(7/8)}$

$= n$

UNIT-2

Binomial Heap:- It is a collection of binomial tree, each of which satisfies regular heap property.

- In this case, we are considering min. heap property.

Binomial Tree:- It is a building block for binomial heap.

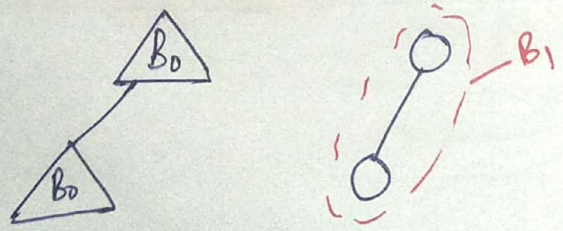
- Binomial tree B_k is an ordered tree defined recursively.

1) B_0 is a single node.

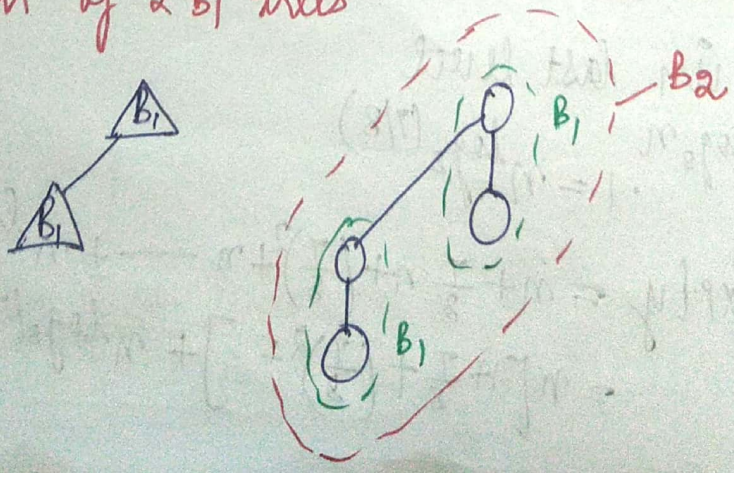
2) Binomial tree B_k consists of 2 binomial trees (B_{k-1}) that are linked together.
Root of one is leftmost child of root of another.

$B_0 = 0$

B_1: collection of 2 B_0 trees



B_2: collection of 2 B_1 trees



Scanned by CamScanner

Properties of Binomial Tree :-

1) There are 2^k nodes.

2) Height of binomial tree is exactly equal to $\log_2 n$.

3) No. of nodes at depth $i = 0$ to k

$$\begin{bmatrix} k \\ l \end{bmatrix} = \frac{k!}{i!\,(k-i)!}$$

$$\begin{bmatrix} 3 \\ 0 \end{bmatrix} = \frac{3!}{0!\,3!} = 1 \qquad \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \frac{3!}{1!\,2!} = 3$$

$$\begin{bmatrix} 3 \\ 2 \end{bmatrix} = \frac{3!}{2!\,1!} = 3 \qquad \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \frac{3!}{3!\,0!} = 1$$

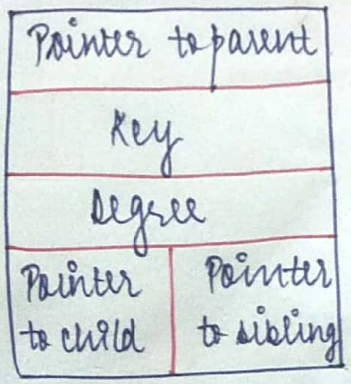
BINOMIAL HEAP :-

Binomial heap (H) is a set of binomial tree that satisfies following properties:-

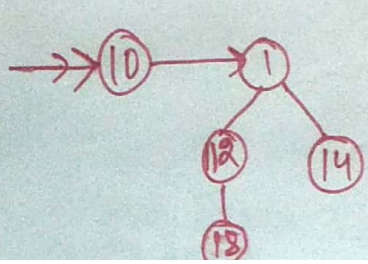
1) Each binomial tree obey min. heap property.

2) There at atmost $\lfloor \log n \rfloor + 1$ binomial trees in binomial heap with n nodes.

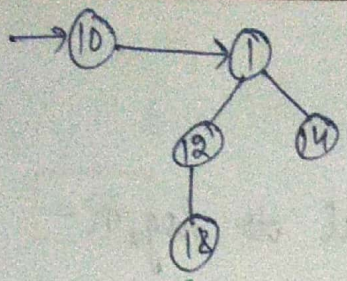
REPRESENTATION OF BINOMIAL HEAP :-



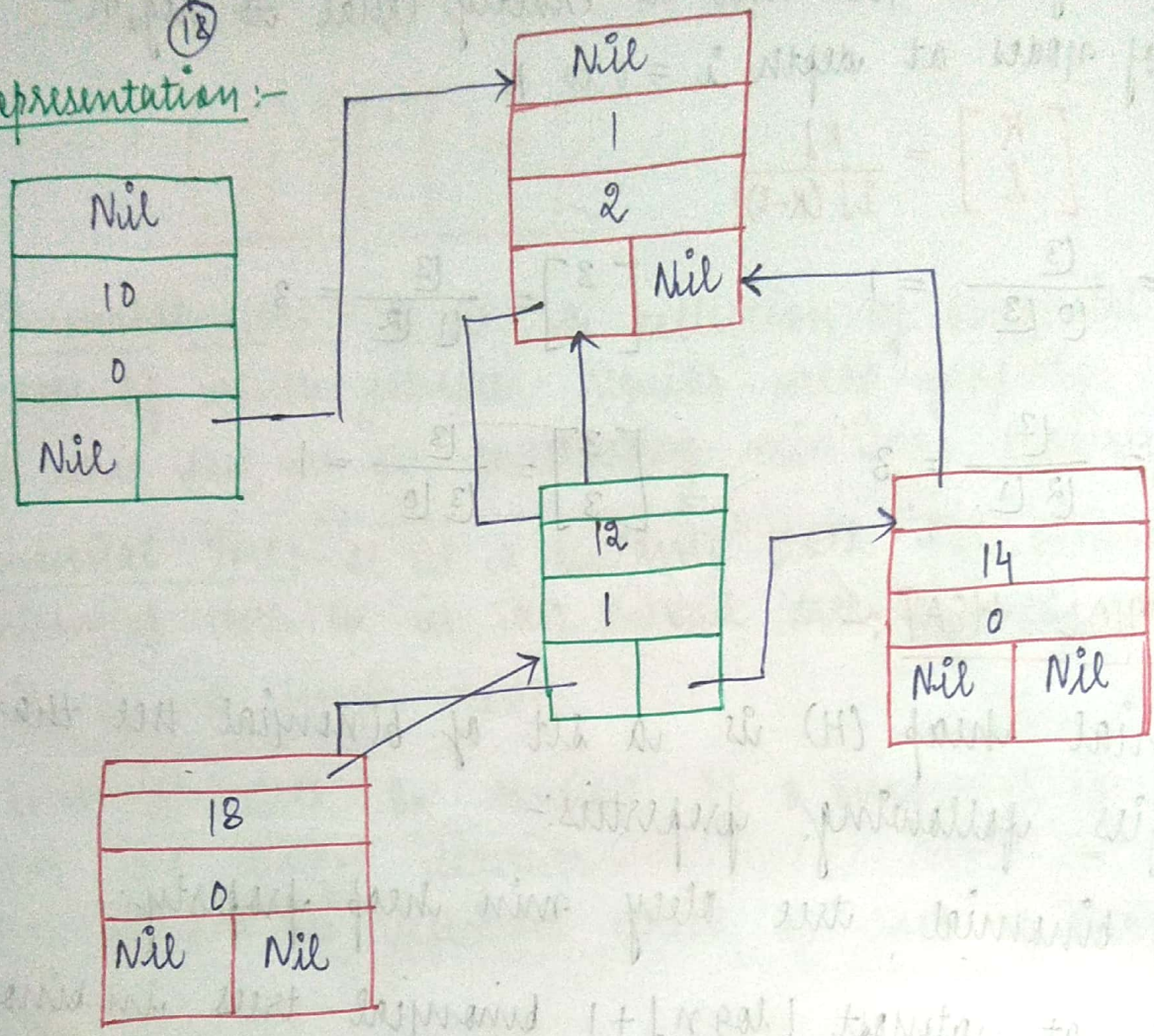
Ex→



Scanned by CamScanner



Representation :-



OPERATIONS ON BINOMIAL HEAP :-

1) Finding minimum key

Binomial Heap min (H)

1) y ← nil
2) x ← Head [H]
3) min ← ∞
4) while x ≠ Nil
5) do if key [x] < min
6) then min ← key [x]

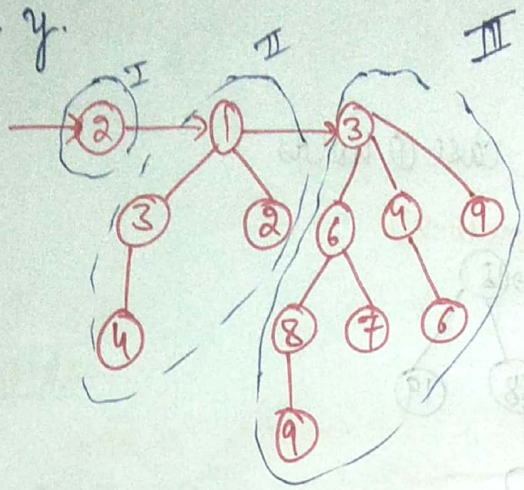
Scanned by CamScanner

7) $y \leftarrow x$

8) $x \leftarrow$ sibling $[x]$

9) return y.

Ex→



2) Union of binomial heap

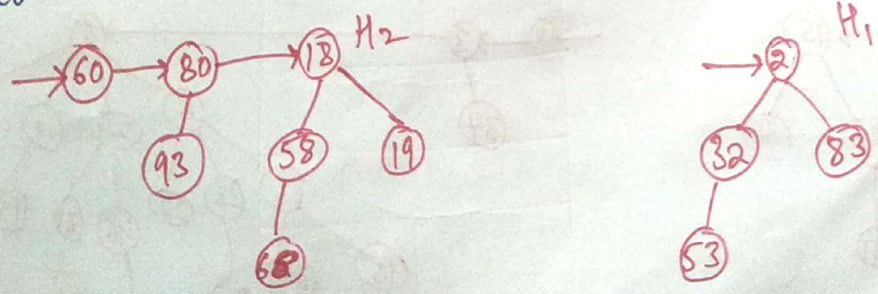
Case 1: If degree $[x] \neq$ degree $[next-x]$, then move the pointers ahead.

Case 2: If degree of $[x]$ = degree $[next-x] =$ degree $[sibling\ [next-x]]$ then move the pointers ahead.

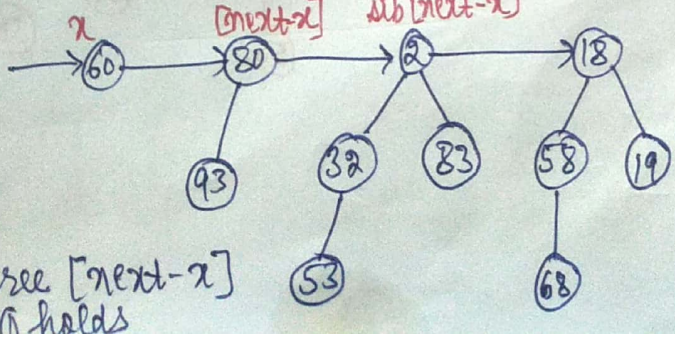
Case 3: If degree $[x]$ = degree $[next-x] \neq$ degree $[sibling\ [next-x]$ and key $[x] \leq$ key $[next-x]$, then remove $[next-x]$ from root list & attach it to x.

Case 4: If degree $[x]$ = degree $[next-x] \neq$ degree $[sibling\ [next-x]$ & key $[x] >$ key $[next-x]$, then remove x from root list & attach it to $[next-x]$.

Ex→



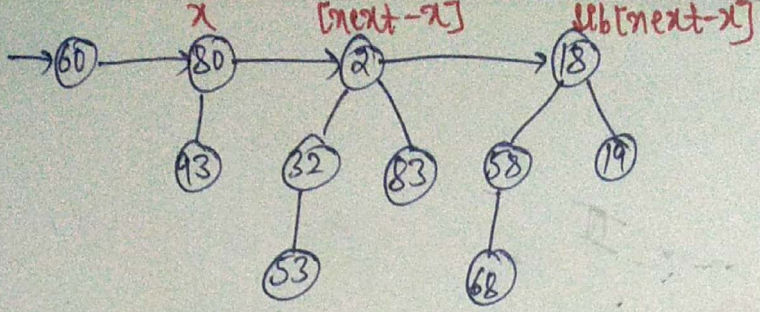
Merging:-



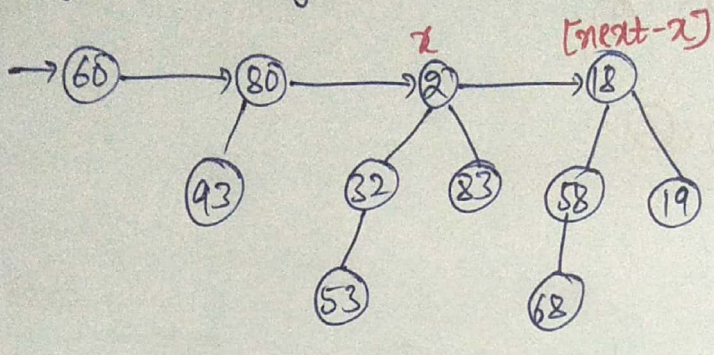
degree $[x] \neq$ degree $[next-x]$

Case 1 holds

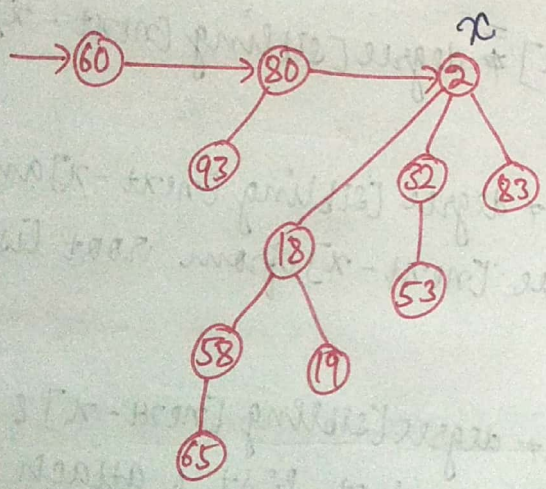
Scanned by CamScanner



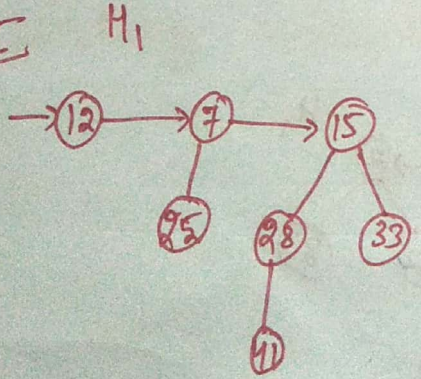
degree[x] ≠ degree[next-x], case ① holds

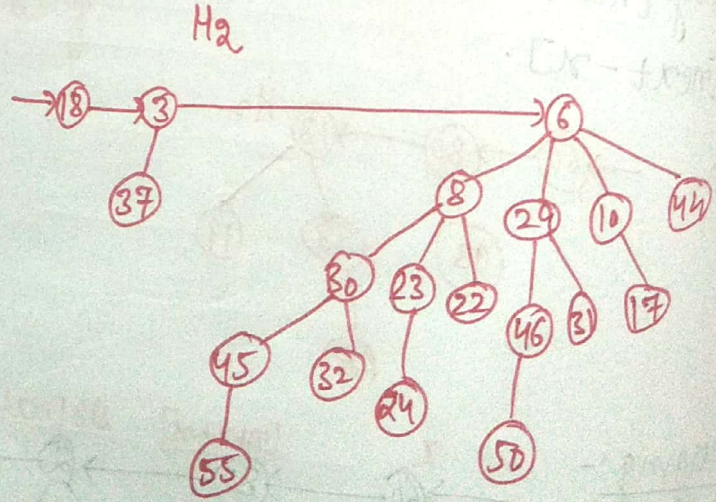


degree[x] = degree[next-x] ≠ degree[sib next-x]
& Key[x] ≤ Key[next-x], case ③ holds:-



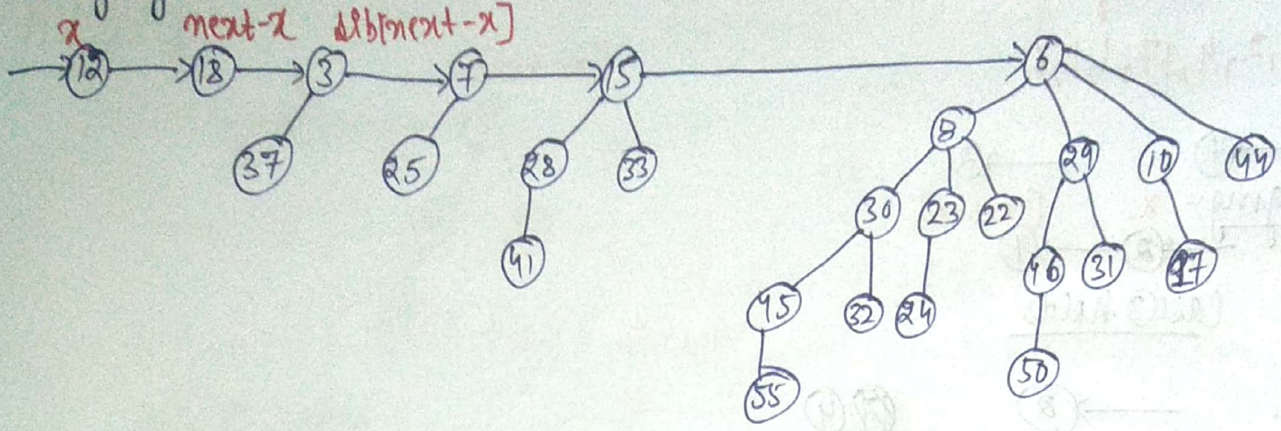
Q:-



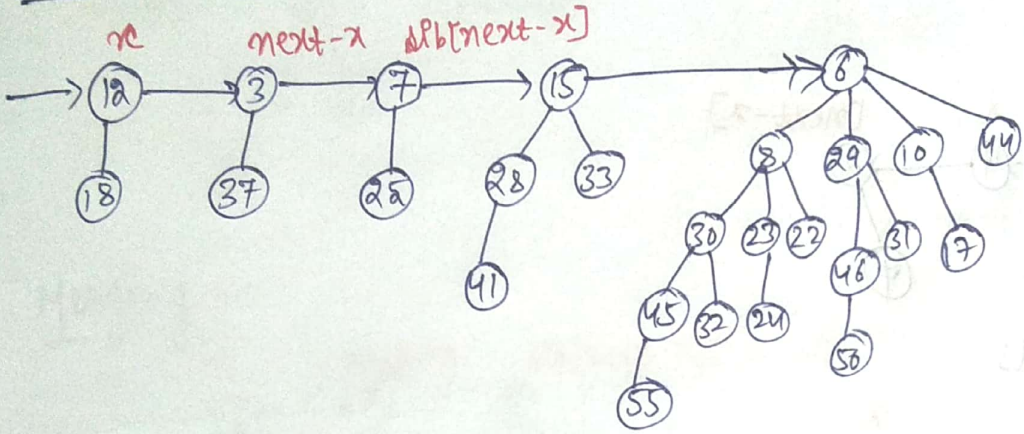


Scanned by CamScanner

Merging :-

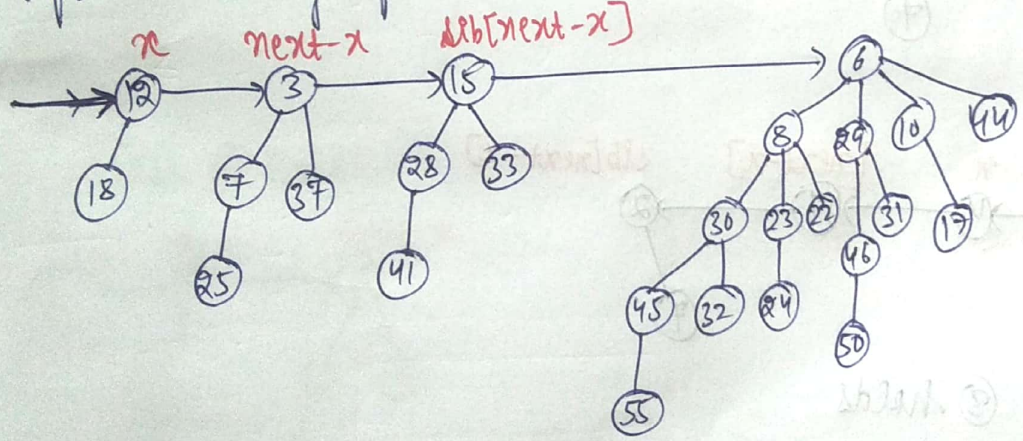


Case ③ holds:-



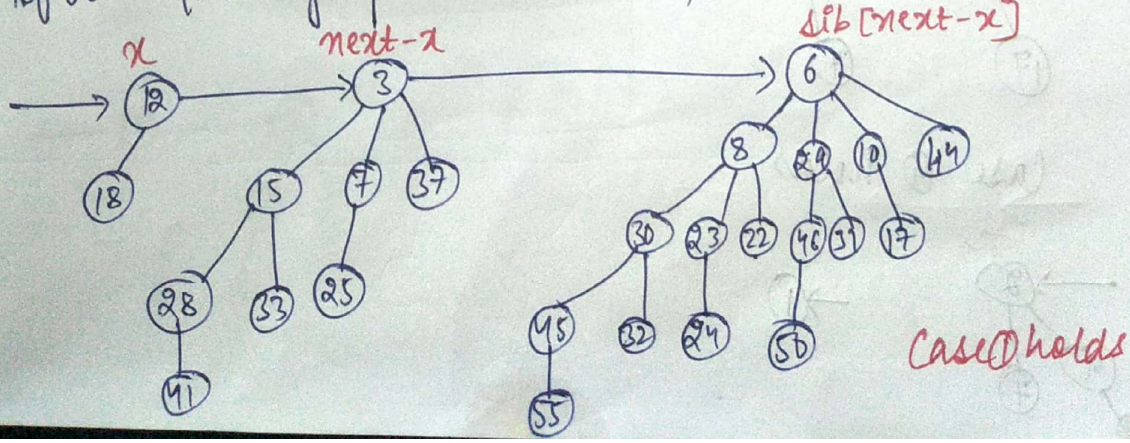
Case 2 holds:-

After moving pointers ahead, case ③ holds



Case ① holds:-

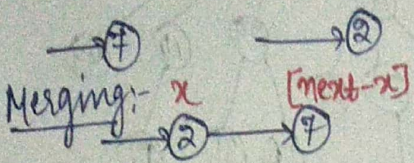
After moving pointer ahead, case ③ holds



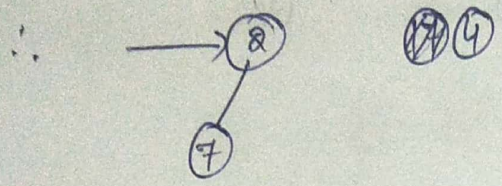
Scanned by CamScanner

Form Heap

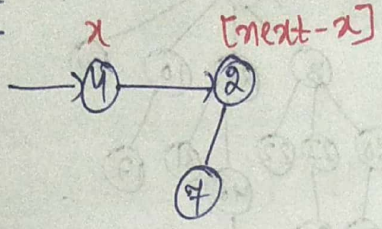
7, 2, 4, 17, 1, 11, 6



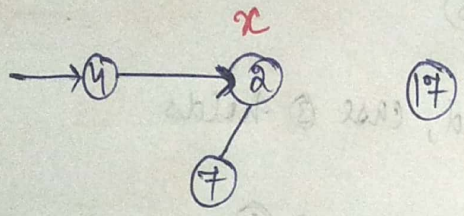
Case ③ holds



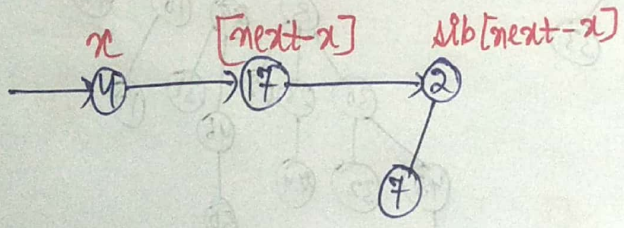
Merging:-



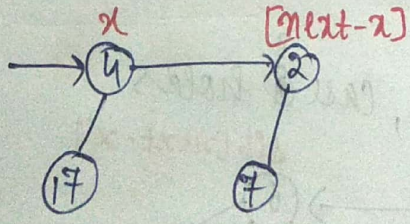
Case ① holds



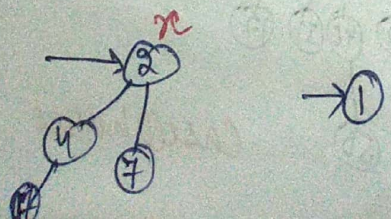
Merging:-



Case ③ holds



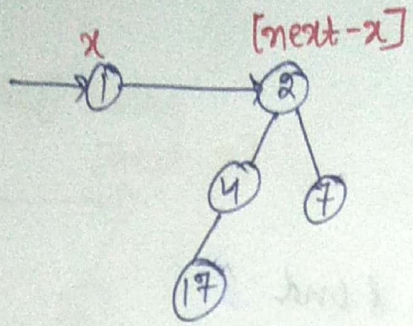
Case ④ holds



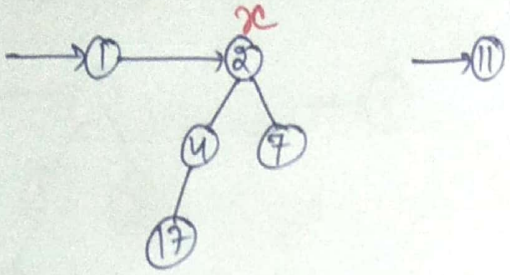
Scanned by CamScanner

case ① hold

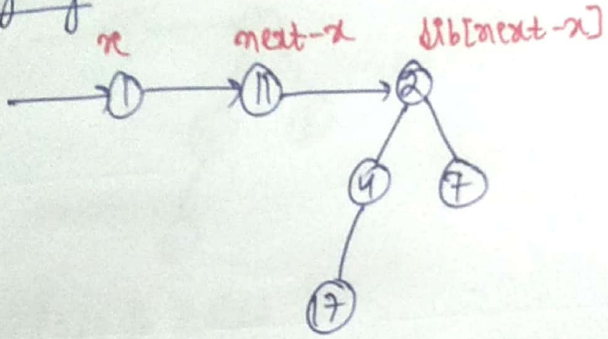
Merging :-



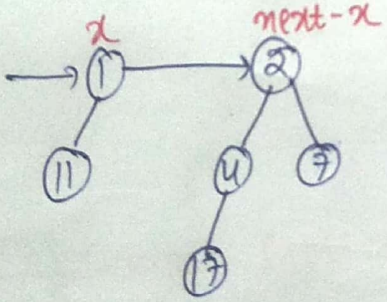
Case ① holds



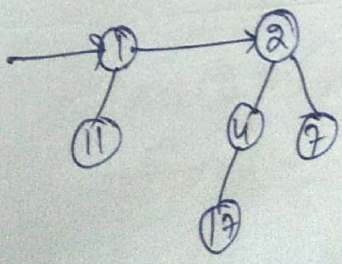
Merging :-



Case ③ holds :-

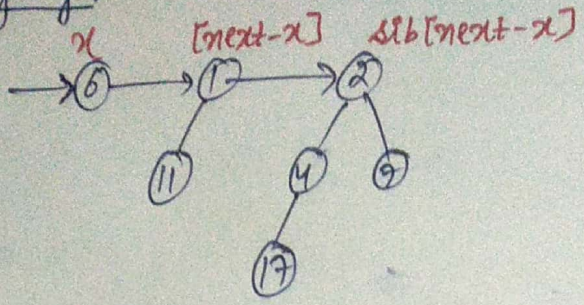


Case ① holds

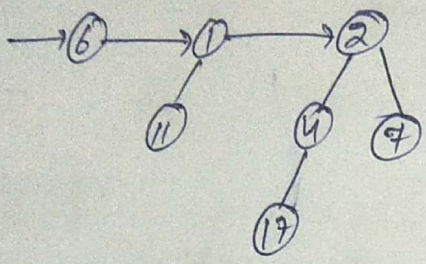


Scanned by CamScanner

Merging :-



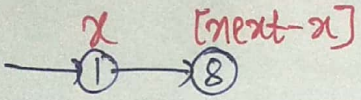
case ① holds, then case ① holds & end



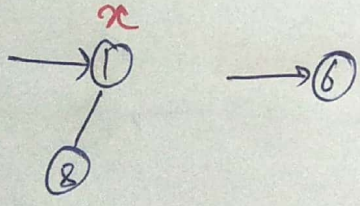
Q:- 8, 1, 6, 5, 7, 2



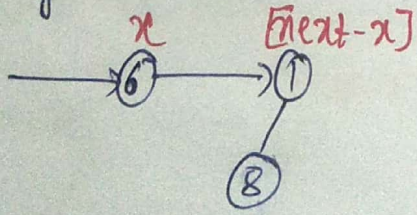
Merging :-



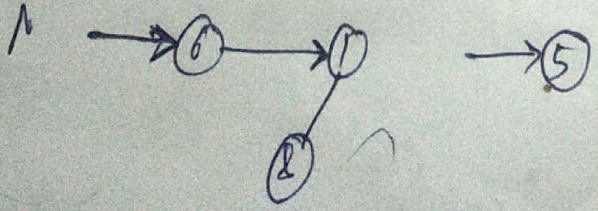
case ③ holds :-



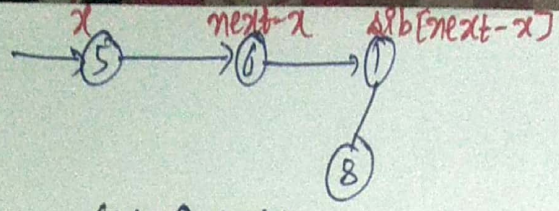
Merging :-



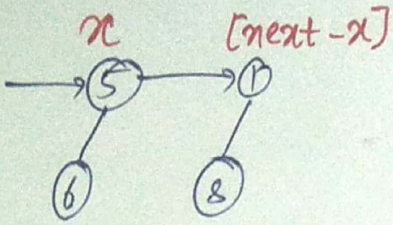
case ① holds



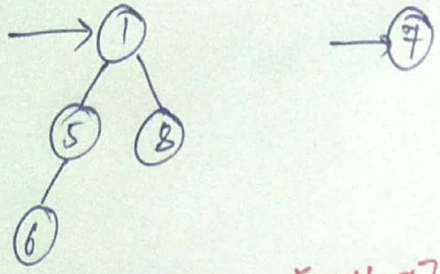
Scanned by CamScanner



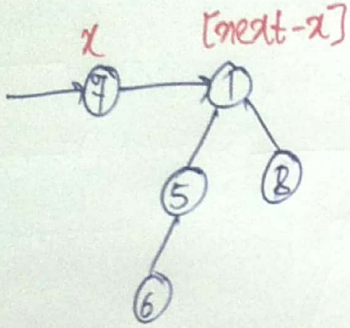
Case ② holds



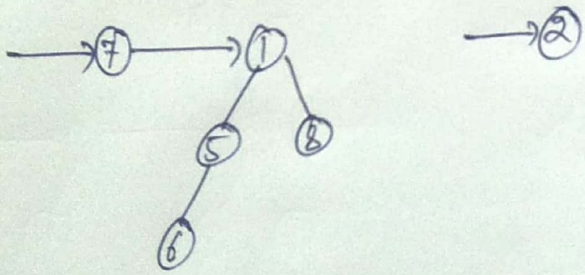
Case ④ holds



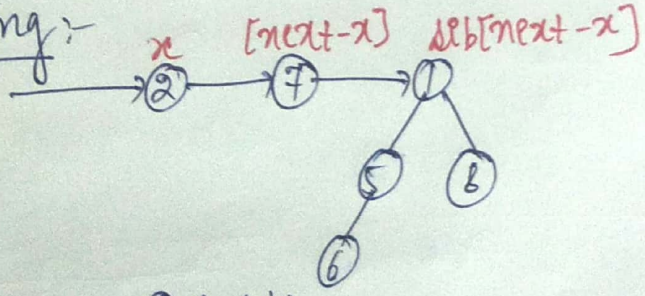
Merging



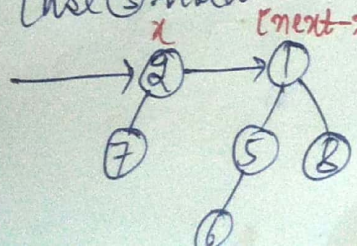
Case ① holds

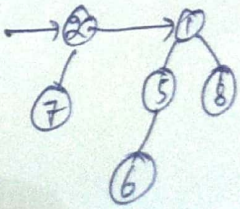


Merging :-



Case ③ holds





Case ① holds (end

Scanned by CamScanner